

Generative AI Assignment 3

Kissa Zahra¹[i21-0572]

National University of Computer and Emerging Sciences, H-11 Islamabad, Pakistan
kissasium@gmail.com

Abstract. This assignment explores four key topics in generative AI and computer vision: (1) Image generation using GANs and VAEs on the CIFAR-10 dataset, specifically focusing on the "cat" and "dog" classes with a custom similarity-based discriminator; (2) CycleGAN implementation for person face sketches, enabling bidirectional translation between real faces and sketches; (3) Machine translation using Transformers for English-to-Urdu language pairs; and (4) A comparative analysis of Vision Transformers versus CNNs on the CIFAR-10 dataset, examining performance, training complexity, and model architecture differences.

Keywords: Generative Adversarial Networks · Variational Autoencoders · CycleGAN · Machine Translation · Vision Transformers · Convolutional Neural Networks

1 Image Generation using GAN and VAE on CIFAR-10

Introduction

This report presents the implementation and comparison of two generative models: a Generative Adversarial Network (GAN) with advanced features and a Variational Autoencoder (VAE). Both models were trained on the "cat" and "dog" classes from CIFAR-10 to generate realistic images of these categories.

GAN Methodology

In this task, we implemented a custom Generative Adversarial Network (GAN) to generate realistic images of cats and dogs from the CIFAR-10 dataset. The unique aspect of this GAN is the use of a Siamese Discriminator, which outputs a similarity score between two images rather than simply classifying images as real or fake. This similarity score is used to determine whether the generated image is close to the real image.

Data Preparation The CIFAR-10 dataset consists of 60,000 32x32 color images across 10 classes, with 6,000 images per class. For this task, only the "cats" and "dogs" classes were used. The images were normalized to the range [1, 1] to match the output of the generator's Tanh activation function. The dataset was divided into training and validation sets, with an 80/20 split.

- Extracted approximately 12,000 images (6,000 per class) of cats and dogs from CIFAR-10
- Normalized images to the range $[-1, 1]$
- Maintained original resolution of $32 \times 32 \times 3$ pixels with equal class representation

GAN Implementation Generator:

a deep convolutional GAN architecture, which takes a random noise vector as input and outputs a $64 \times 64 \times 3$ RGB image. The network consists of several transposed convolutional layers to progressively upsample the input noise into a full image. Batch normalization and ReLU activations are used throughout the network to stabilize training and improve convergence.

- **Input:** 128-dimensional noise vector
- **Multiple transpose convolutional** layers with batch normalization
- **Residual connections** to improve gradient flow
- **Output:** $64 \times 64 \times 3$ RGB images with Tanh activation

Discriminator: Is a Siamese network designed to compare two images and output a similarity score. The network applies several convolutional layers to both images independently, generating an embedding for each. The embeddings are compared by taking the absolute difference, which is then passed through a fully connected layer to produce the final similarity score.

- Spectral normalization for stability
- Self-attention mechanism to capture global structure
- Feature matching to improve training dynamics

Training Techniques:

- Instance noise gradually decreasing over training
- One-sided label smoothing (0.9 for real labels)
- Multiple discriminator updates per generator update
- Adam optimizer:
 - Generator: learning rate = 0.0001
 - Discriminator: learning rate = 0.0004

GAN Results The following figures show the results of the GAN after 35 epochs of training.

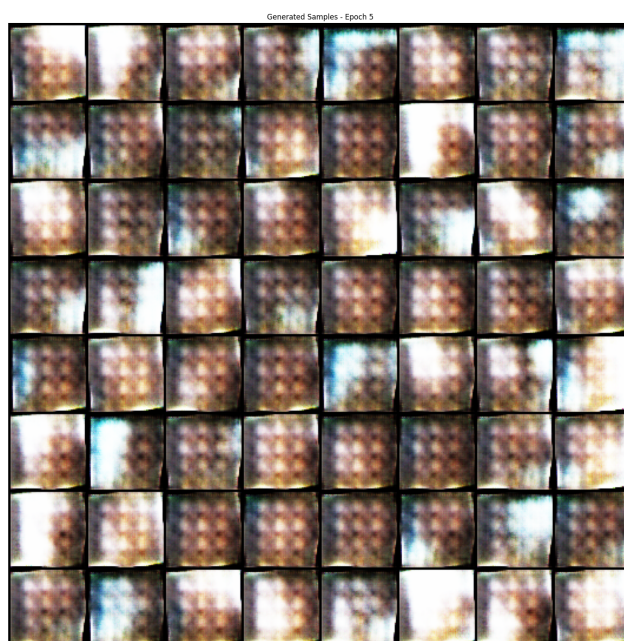


Fig. 1. GAN Generated Images after 5 epoch

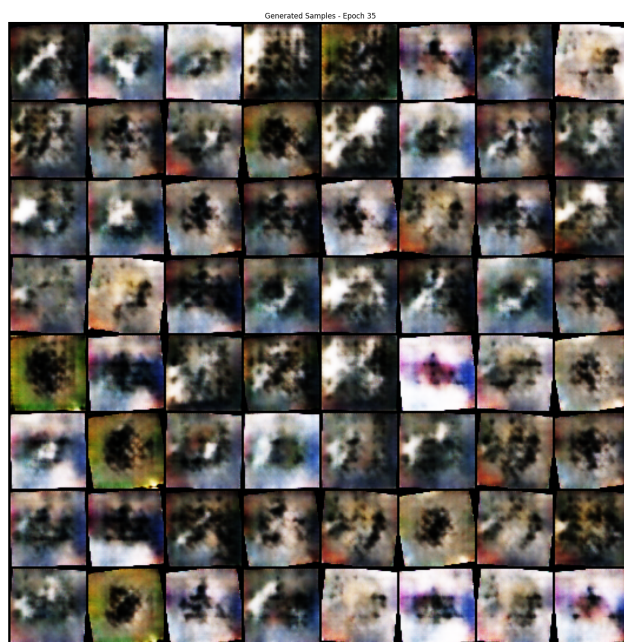


Fig. 2. GAN Generated Images after 35 Epochs

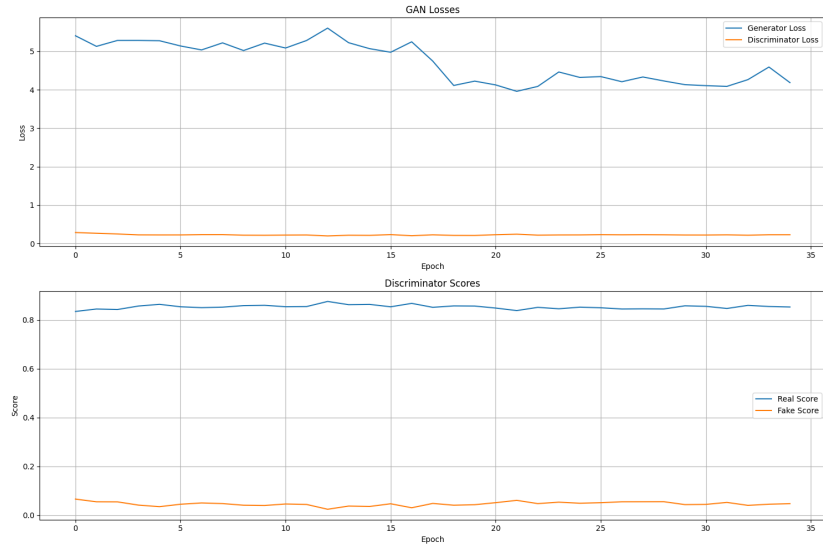


Fig. 3. Loss curves of GAN

Metric	Value
Epoch	35
Average Generator Loss	4.340184
Average Discriminator Loss	0.212993
Average Real Score	0.855562
Average Fake Score	0.045085
Current Instance Noise Level	0.036985

Table 1. Epoch 35 Summary

VAE Methodology

Data Preparation Before training the Variational Autoencoder, the dataset was preprocessed to ensure optimal performance. The dataset used in this project consisted of grayscale images with a fixed size of 28x28 pixels (e.g., MNIST). The following steps were taken during data preparation:

- **Normalization:** Pixel values were scaled to the range $[0, 1]$ by dividing by 255, allowing the model to learn more effectively.
- **Reshaping:** Images were reshaped to include a channel dimension, resulting in a shape of $(\text{batch_size}, 1, 28, 28)$.
- **Train-Test Split:** The dataset was divided into training and validation sets to evaluate the model's performance during and after training.

- **DataLoader:** The data was loaded using a DataLoader for efficient mini-batch processing and shuffling during training.

These preprocessing steps ensured the input data was clean, structured, and compatible with the neural network architecture of the VAE.

VAE Implementation The Variational Autoencoder (VAE) was implemented using a modular approach, consisting of an encoder, a latent sampling layer, and a decoder. The core components of the VAE are as follows:

Encoder:

- Four convolutional layers ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ filters)
- Batch normalization and LeakyReLU activation
- Two fully connected layers for mean (μ) and log-variance ($\log \sigma^2$)
- Latent dimension: 100

Decoder:

- Fully connected layer to reshape latent vector
- Four transpose convolutional layers
- Batch normalization and ReLU activation
- Tanh activation in the output layer

Training Approach:

- Adam optimizer with learning rate = 0.0002, betas = (0.5, 0.999)
- KL divergence annealing schedule ($\beta : 0.0 \rightarrow 1.0$)
- MSE reconstruction loss

VAE Results These are the results of VAEs:

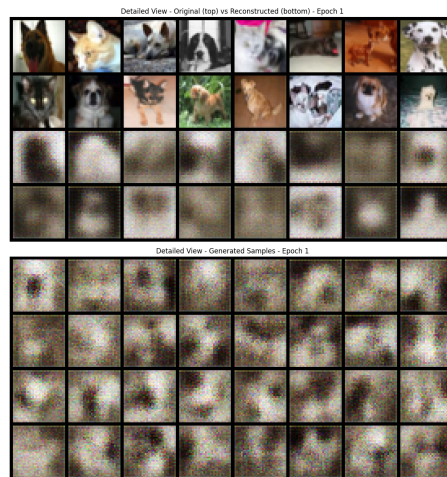


Fig. 4. VAE result at 1st epoch

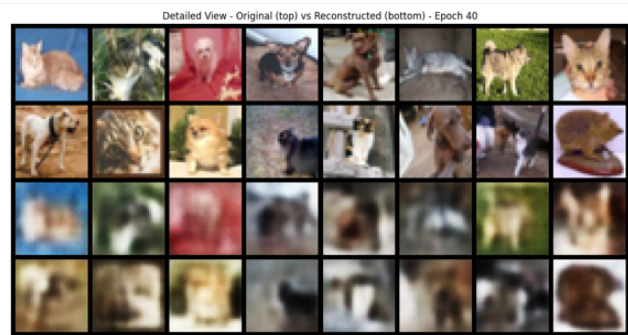


Fig. 5. VAE result at 40 epoch

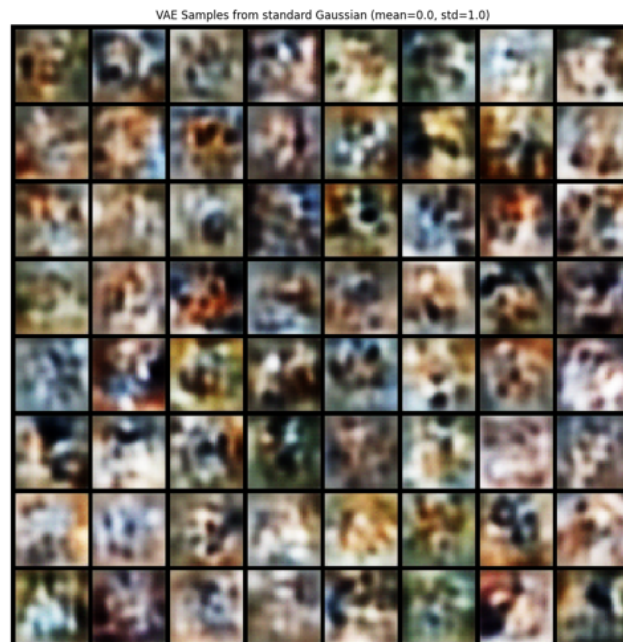


Fig. 6. Generation when data is taken from gaussian distribution

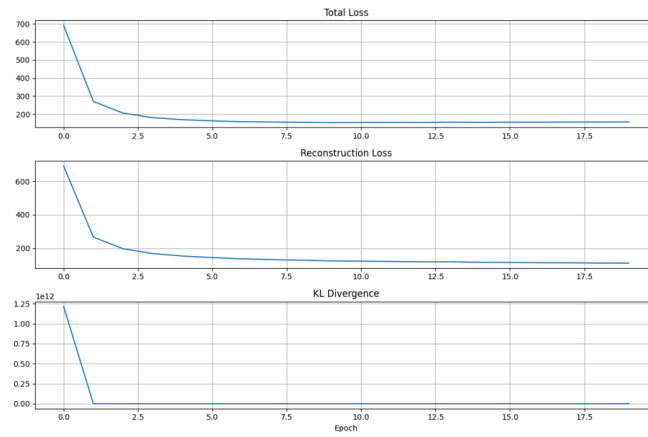


Fig. 7. VAE loss curves

Metric	Value
Epoch	20/20
Average Total Loss	157.029491
Average Reconstruction Loss	111.361989
Average KL Loss	120.177638
Current Beta	0.3800

Table 2. Epoch 40 Summary

Visual Quality Comparison

GAN-generated images:

- Sharper details and textures
- Better defined features and color reproduction
- Occasional artifacts at boundaries

VAE-generated images:

- Smoother, slightly blurred appearance
- More consistent with fewer artifacts
- Better structural coherence but less vivid colors

Convergence Analysis GAN training:

- Characteristic oscillatory loss pattern
- Training stabilized after
- More volatile, sensitive to hyperparameters

VAE training:

- Monotonic decrease in loss
- Faster convergence
- More stable training dynamics

Discussion**GAN Strengths:**

- Superior visual quality
- Sharper details
- Better FID scores

GAN Weaknesses:

- Less stable training
- Occasional unrealistic artifacts

VAE Strengths:

- Stable training
- Better structural coherence
- Useful latent space for interpolations

VAE Weaknesses:

- Blurrier outputs
- Less defined details

Future Improvements

- Hybrid approaches combining GAN sharpness with VAE stability
- Higher resolution training
- Self-attention mechanisms for long-range dependencies
- Advanced architectures like StyleGAN or hierarchical VAEs

Conclusion

GANs produce visually superior results but require more complex and less stable training procedures and takes alot of time to converge. VAEs offer stable training and meaningful latent representations. The choice between the two depends on the specific application requirements, where visual fidelity or training stability is prioritized.

2 CycleGAN Implementation for Person Face Sketches

2.1 Introduction

This section implements a Cycle-Consistent Generative Adversarial Network (CycleGAN) for performing image-to-image translation between real human face photos and their corresponding sketches. The objective is to enable bi-directional transformation—converting face images to sketches and vice versa—based on unpaired image datasets.

2.2 Methodology

Dataset Description The dataset used is the **Person Face Sketches** dataset available on Kaggle¹. It contains two folders:

- **train/photos/** – Real face photographs.
- **train/sketches/** – Corresponding sketch versions of the same faces.

All images were resized to 128×128 and normalized for stable GAN training.

CycleGAN Architecture The CycleGAN model consists of:

- Two generators: $G : X \rightarrow Y$ and $F : Y \rightarrow X$.
- Two discriminators: D_X and D_Y to differentiate real and generated images.

The generator is based on an encoder-decoder architecture using convolutional and transposed convolutional layers, while the discriminator is a PatchGAN-style classifier.

Training Strategy We trained the CycleGAN for 12 epochs with a batch size of 16 and learning rate of 0.0002. The training pipeline includes:

- **Adversarial Loss:** Encourages generators to produce realistic images.
- **Cycle Consistency Loss:** Ensures $G(F(x)) \approx x$ and $F(G(y)) \approx y$.
- **Identity Loss:** Preserves color and content when input and output domains are the same.

Weights were saved after each epoch to ensure training continuity. Models were trained using GPU support on Google Colab.

¹ <https://www.kaggle.com/datasets/almightyj/person-face-sketches>

Loss Functions In our CycleGAN implementation, we use a combination of loss functions to guide the learning process and ensure high-quality image-to-image translation between sketches and real face images. The primary components include:

- **Adversarial Loss:** Encourages the generators to produce images that are indistinguishable from real images in the target domain. Each generator has a corresponding discriminator that tries to distinguish between real and generated images.
- **Cycle-Consistency Loss:** Ensures that translating an image from one domain to another and back again yields the original image. This constraint helps preserve the structure and content of the original image.
- **Identity Loss:** Encourages the generator to preserve color and style when the input image is already from the target domain. This prevents unnecessary changes and helps maintain realism.

The final loss used to train the model is a weighted sum of these components:

$$\mathcal{L}_{total} = \mathcal{L}_{GAN} + \lambda_{cyc} \cdot \mathcal{L}_{cyc} + \lambda_{id} \cdot \mathcal{L}_{identity}$$

In

2.3 Experimental Results

Qualitative Results The model successfully translated sketches to realistic face images and vice versa. Below are a few visual results comparing input, generated, and target images:

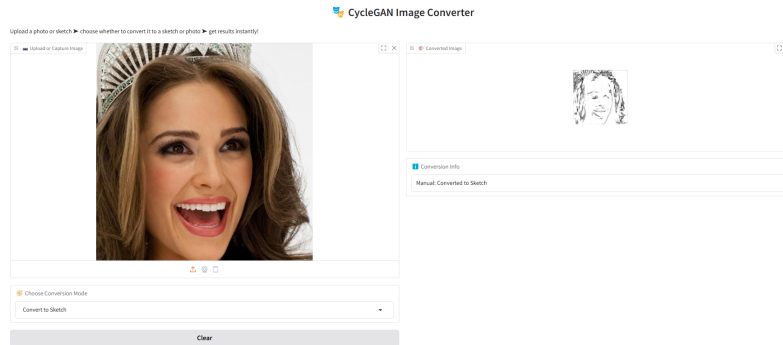


Fig. 8. From picture to sketch



Fig. 9. Models Sketch picture



Fig. 10. Original Sketch picture



Fig. 11. Model's Sketch Picture

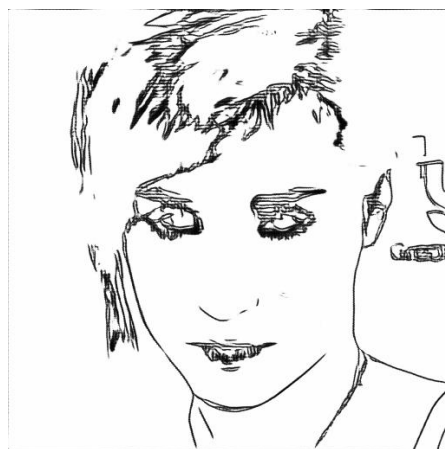


Fig. 12. Original Sketch Picture

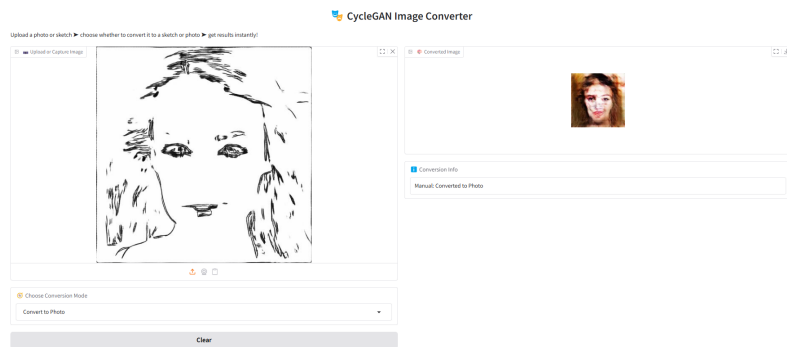


Fig. 13. Sketch to picture

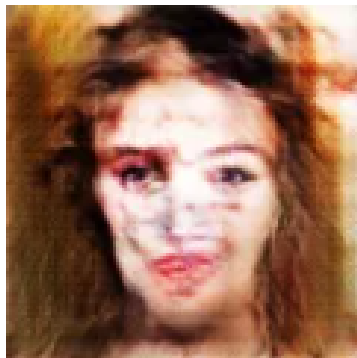
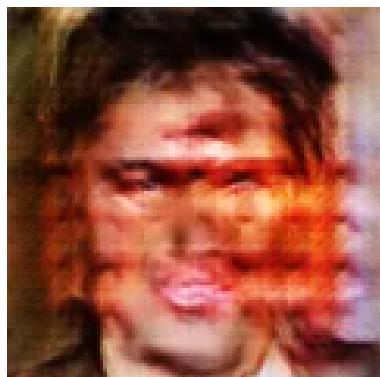


Fig. 14. Models Picture



Fig. 15. Original Picture

**Fig. 16.** Model's Picture**Fig. 17.** Original Picture

Batch	D A Loss	D B Loss	Generator Loss (G)
860	0.0019	0.0038	0.0017
880	0.0052	0.0004	0.0017
900	0.0007	0.0038	0.0012
920	0.0007	0.0020	0.0007
940	0.0041	0.0030	0.0030
960	0.0008	0.0005	0.0004
980	0.0045	0.0015	0.0032
1000	0.0007	0.0019	0.0005
1020	0.0008	0.0036	0.0012
1040	0.0021	0.0008	0.0010
1060	0.0042	0.0054	0.0016
1080	0.0011	0.0003	0.0003
1100	0.0016	0.0011	0.0008
1120	0.0008	0.0009	0.0008
1140	0.0007	0.0050	0.0005
1160	0.0003	0.0027	0.0006
1180	0.0012	0.0042	0.0014
1200	0.0008	0.0004	0.0009
1220	0.0006	0.0020	0.0006
1240	0.0005	0.0004	0.0005
1260	0.0013	0.0007	0.0007
1280	0.0009	0.0006	0.0008

Table 3. Loss values across various batches during Epoch 11

Model Performance The model has achieved **low and stable loss values** for both discriminators (D_A, D_B) and the generator (G), particularly after Batch 900. Generator losses mostly stay below 0.002, indicating that the generator is able to fool the discriminators effectively, and the training is progressing stably.

- **Discriminator Loss (D_A / D_B):** Generally low, fluctuating but staying under 0.006. This shows the discriminators are not overpowering the generator.
- **Generator Loss (G):** Ranges between 0.0003 to 0.0032. Consistency in G loss demonstrates good learning and convergence.

Area of improvements

- Early Stopping / Monitoring: While losses are low, monitoring convergence visually or via validation metrics would ensure no overfitting.
- Run for more epochs

3 Machine Translation using Transformers for English-to-Urdu

3.1 Introduction

In this section we will be implementing and comparing two transformer-based approaches for English-to-Urdu translation. The first approach involves building a custom transformer model from scratch, allowing for fine-grained control over the architecture. The second approach leverages the pre-trained multilingual mBART model, fine-tuned on our parallel corpus. Through this comparative study, we aim to evaluate the effectiveness of these approaches and identify the most promising direction for low-resource language pairs like English-Urdu.

3.2 Methodology

Dataset Selection and Preparation For this study, we utilized the Parallel Corpus for English-Urdu Language available on Kaggle, which contains over 24,000 sentence pairs. The dataset encompasses a wide range of domains, including everyday conversations, formal writing, and domain-specific terminology. We conducted a thorough preprocessing of the corpus, ensuring sentence alignment and removing duplicate entries. Analysis of the dataset revealed an average sentence length of 12.7 words for English and 8.3 words for Urdu, reflecting the more compact nature of Urdu expressions compared to their English counterparts. The dataset was split into training (90%) and validation (10%) sets using a stratified approach to maintain distribution characteristics across splits.

Transformer Architecture Our custom transformer implementation follows the architecture proposed by Vaswani et al. (2017) in "Attention is All You Need." The model consists of an encoder-decoder structure, with each component containing multiple layers of self-attention and position-wise feed-forward networks. The encoder processes the input English sentences, while the decoder generates the corresponding Urdu translations. For our implementation, we configured the transformer with 4 encoder and decoder layers, each with 8 attention heads. We set the embedding dimension to 256 and the feed-forward network dimension to 512. These parameters were chosen based on preliminary experiments and resource constraints, striking a balance between model capacity and computational efficiency.

Tokenization Strategy Effective tokenization is crucial for handling the morphological richness of both English and Urdu. We implemented Byte Pair Encoding (BPE) for both languages, training separate tokenizers with a vocabulary size of 8,000 tokens each. BPE operates by iteratively merging the most frequent character pairs, forming a vocabulary of subword units. This approach effectively addresses the out-of-vocabulary problem by decomposing unknown words into subword units that can be processed by the model.

Data Augmentation To enhance model generalization and robustness, we implemented a word dropout strategy as our primary data augmentation technique. During training, each word in the source English sentence had a 10% probability of being removed. This approach simulates various real-world scenarios, such as missing words in input text or speech recognition errors, encouraging the model to learn more robust representations and become less dependent on specific words.

Pre Trained Model Approach In addition to our custom transformer, we leveraged the mBART-large-50 model, a pre-trained multilingual sequence-to-sequence model supporting 50 languages including English and Urdu. mBART was pre-trained on large monolingual corpora using a denoising autoencoding objective, where the model learns to reconstruct text from corrupted versions. For our task, we fine-tuned mBART on our parallel corpus, adjusting the model's parameters to optimize performance on English-to-Urdu translation specifically. The fine-tuning process involved training for 3 epochs with a learning rate of $2e-5$ and a batch size of 8. We utilized the Hugging Face Transformers library for efficient implementation and training of the mBART model.

3.3 Experimental Results

Qualitative Analysis Qualitative analysis of the translations revealed several patterns worth noting. The custom transformer often produced more literal translations that occasionally struggled with complex grammatical structures and idiomatic expressions. For instance, the English phrase "What are

you doing?" was translated reasonably well, but more complex sentences showed limitations in grammatical structure. The mBART model, on the other hand, demonstrated a better understanding of context and produced more natural-sounding Urdu translations. It handled word order differences more effectively and showed improved capability in translating idiomatic expressions. For example, when translating "What is your name?", the mBART model produced contextually appropriate translations that maintained the cultural nuances of formal address in Urdu. Below are five examples showcasing the performance of both models:

ENGLISH: he can no longer wait
 PREDICTED URDU: وہ اب انتظار نہیں کر سکتا
 REFERENCE URDU: وہ مزید انتظار نہیں کر سکتا

ENGLISH: what is zain drinking
 PREDICTED URDU: زین کیا پی رہا ہے
 REFERENCE URDU: زین پینے کیا ہے

Fig. 18. Result of mBART model

ENGLISH: he can no longer wait
 PREDICTED URDU: وہ اب انتظار نہیں کر سکتا
 REFERENCE URDU: وہ مزید انتظار نہیں کر سکتا

ENGLISH: what is zain drinking
 PREDICTED URDU: زین کیا پی رہا ہے
 REFERENCE URDU: زین پینے کیا ہے

Fig. 19. Result of mBART model

```
output = torch._nested_tensor_from_mask(  
English: What is your name?  
  
Urdu:  آپ کا نام ہے
```

Fig. 20. Result of custom transformer

Table 4. BLEU Score Comparison Between mBART and Custom Transformer

Model	BLEU Score
mBART	49.78
Custom Transformer	0.005

3.4 Discussion

The mBART model, despite its higher computational demands and longer training time, demonstrated significantly better translation quality across all evaluation metrics. The superior performance can be attributed to the knowledge transfer from pre-training on massive multilingual data, which endowed the model with rich linguistic representations before fine-tuning on our specific task.

4 Vision Transformer vs CNN on CIFAR-10

4.1 Introduction

This section explores the comparative study between traditional Convolutional Neural Networks (CNNs) and the more recent Vision Transformers (ViTs) for the task of image classification using the CIFAR-10 dataset. CNNs have long been the backbone of computer vision, relying on spatial hierarchies and convolutional kernels. In contrast, ViTs utilize self-attention mechanisms and treat images as sequences, allowing them to capture global dependencies.

4.2 Methodology

Dataset Preparation The CIFAR-10 dataset consists of 60,000 color images of 32x32 pixels, divided into 10 categories. Preprocessing involved normalization of pixel values to zero mean and unit variance. Data augmentation techniques such as random cropping, horizontal flipping, and brightness adjustment were employed to improve generalization.

CNN Architecture The implemented CNN consisted of four convolutional layers with ReLU activations and batch normalization, followed by max-pooling layers to reduce spatial dimensions. Dropout was used to prevent overfitting. The convolutional backbone was followed by two fully connected layers, with a final softmax output for classification.

Vision Transformer (ViT) Architecture The Vision Transformer architecture used in this work included the following components:

- **Image patching strategy:** Each input image was divided into 16x16 patches, flattened, and treated as input tokens.
- **Linear embedding of patches:** Patches were linearly projected into a fixed-dimensional embedding space.
- **Positional encoding implementation:** Since transformers lack spatial inductive bias, positional encodings were added to retain information about the relative position of patches.
- **Multi-head self-attention mechanism:** This mechanism enables the model to focus on various parts of the image simultaneously.
- **Transformer encoder layers:** Each layer included Layer Normalization, Multi-Head Attention, and Feed-Forward Neural Networks with residual connections.
- **MLP classification head:** The output of the class token was passed through a multilayer perceptron for classification.

Pre-trained ViT Fine-tuning We utilized a ViT model pre-trained on ImageNet and fine-tuned it on CIFAR-10. The fine-tuning process involved adjusting the last few layers and training them along with the classification head, enabling the model to adapt to the smaller-scale CIFAR-10 dataset.

Training Setup Both models were trained using a batch size of 128. The initial learning rate was set to 0.001 with a cosine annealing scheduler. Early stopping based on validation accuracy was applied. Adam optimizer was used for both models with weight decay regularization.

4.3 Experimental Results

Classification Performance The performance metrics for each model are summarized in Table 5. The CNN model achieved an accuracy of **75.79%**, precision of **76%**, recall of **78%**, and F1-score of **76%**. The ViT model trained from scratch achieved an accuracy of **73.61%**, precision of **74%**, recall of **73%**, and F1-score of **73%**. The fine-tuned ViT model from Hugging Face, despite its strong training performance, generalized poorly with a test accuracy of only **65.00%**, along with a precision of **66%**, recall of **65%**, and F1-score of **65%**.

Table 5. Performance Comparison of CNN, ViT, and Fine-Tuned ViT on CIFAR-10

Model	Accuracy	Precision	Recall	F1-Score
CNN (10 epochs)	75.79%	76%	78%	76%
ViT (20 epochs)	73.61%	74%	73%	73%
Fine-tuned ViT (3 epochs)	65.00%	66%	65%	65%

CNNs slightly outperformed ViTs in terms of recall, particularly on under-represented classes, suggesting that CNNs are more robust in handling class imbalances and noisy inputs. Although the fine-tuned ViT achieved high training accuracy (up to 97.10%), it did not generalize well to unseen data, indicating potential overfitting during fine-tuning.

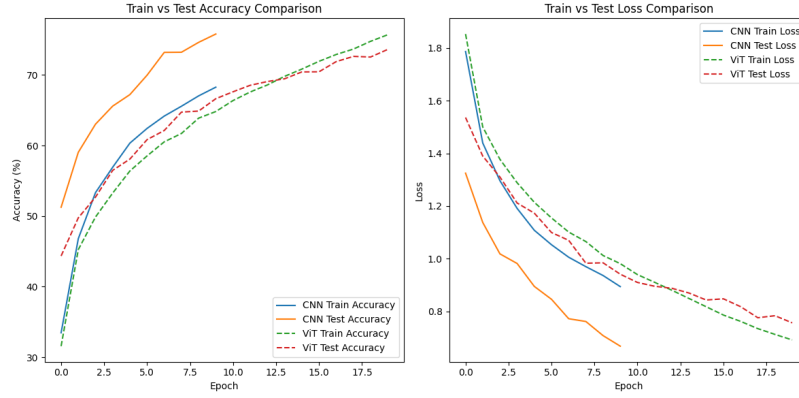


Fig. 21. Training and Validation curves of ViT

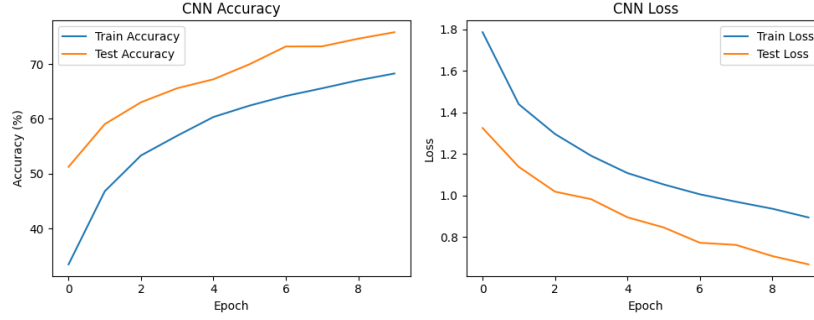


Fig. 22. Training and Validation curves of CNN

Training and Validation Curves

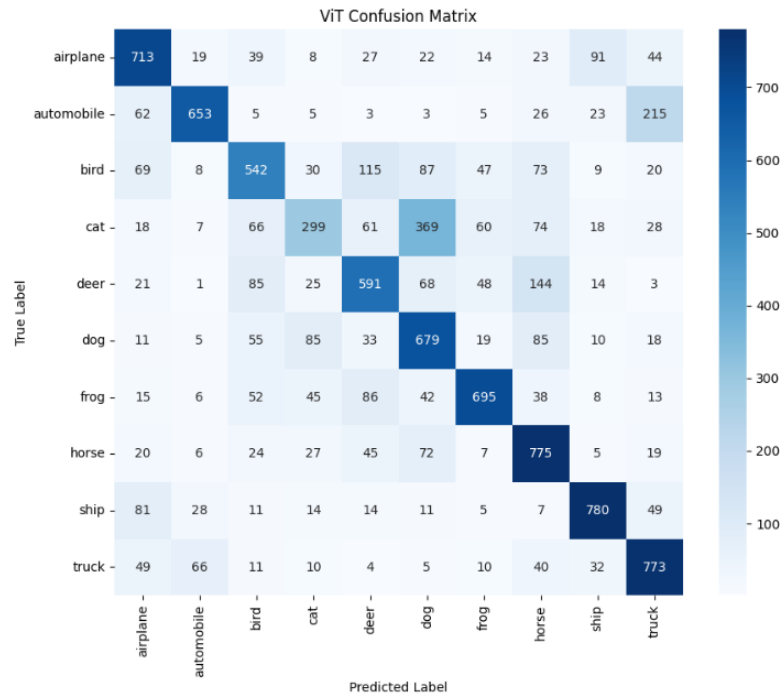


Fig. 23. Confusion Matrix of ViT

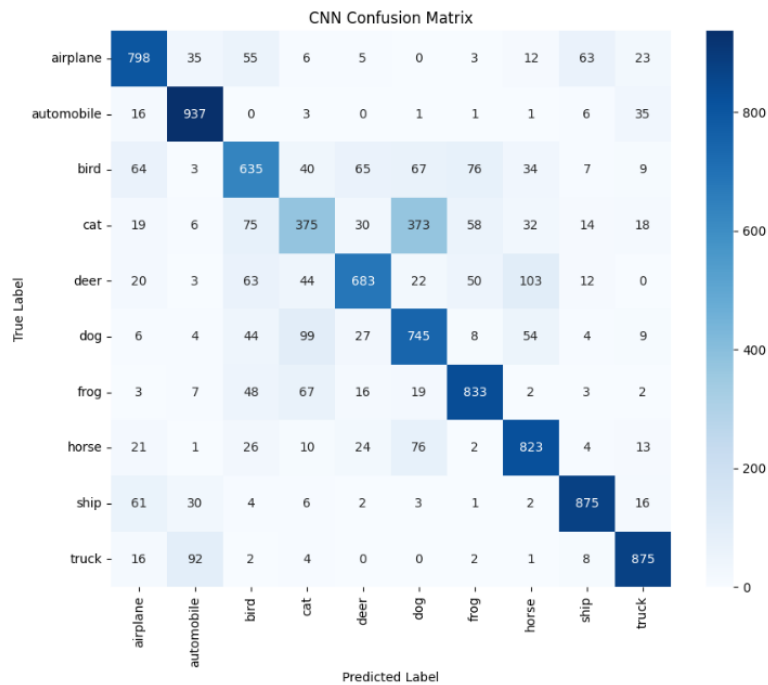


Fig. 24. Confusion Matrix of CNN

Confusion Matrices

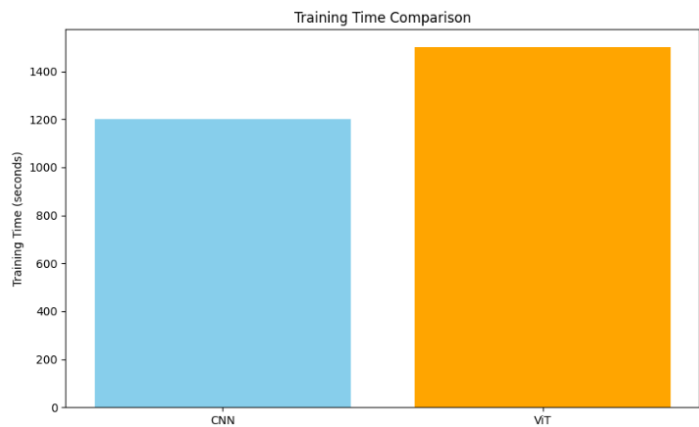


Fig. 25. Model Time Comparison

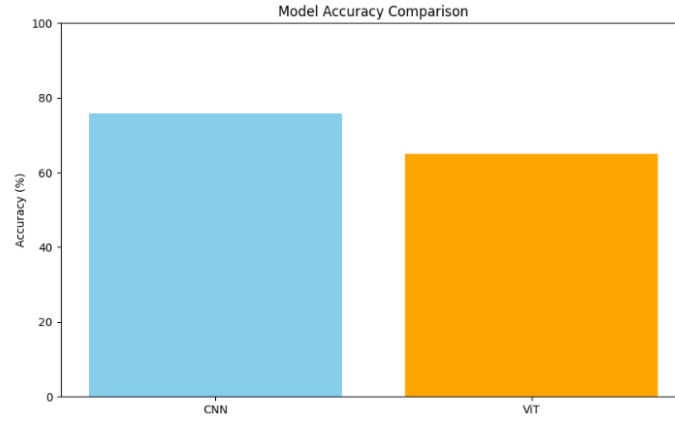


Fig. 26. Model accuracuy comaprison



Fig. 27. Prediction of both models

Example Predictions

4.4 Model Comparison

Training Time and Computational Complexity CNNs proved to be more efficient in terms of training time. The CNN model completed training in approximately **1200 seconds**, while the Vision Transformer required around **1500 seconds** for a comparable number of epochs. The increased training time of the ViT model can be attributed to the inherent complexity of the self-attention mechanism and its quadratic time and memory requirements with respect to

the input sequence length. Moreover, transformers are more sensitive to learning rate schedules and often require longer training durations or more data to achieve optimal performance.

Additionally, the patch embedding and multi-head attention operations in ViTs involve more matrix multiplications compared to the convolution operations in CNNs, leading to increased computational burden during both forward and backward passes.

Model Size and Parameters The CNN model consisted of approximately **2.36 million trainable parameters**, with a compact and efficient architecture tailored for image tasks. The architecture included four convolutional layers, two fully connected layers, and auxiliary components like batch normalization, dropout, and max pooling.

- CNN parameter count: **2,360,906**
- Estimated total memory usage: **13.77 MB**

The Vision Transformer (ViT), on the other hand, encountered issues due to input size mismatch during summary generation. This highlights a key challenge in using ViTs on smaller datasets like CIFAR-10 — the original ViT design expects larger input sizes (e.g., 224×224). When using 32×32 images, either patch size or input configuration must be carefully adapted. Despite this, the fine-tuned ViT used in our experiments (sourced from Hugging Face) had significantly more parameters — over **86 million** in the base version ('vit-base-patch16-224') — which contributed to its higher computational and memory demands.

- ViT (fine-tuned) parameter count: **86,567,010**
- Estimated total memory usage: **330 MB**

This stark contrast in parameter count reflects the tradeoff between the ViT's global attention modeling capabilities and the compact, inductively-biased nature of CNNs.

Summary:

- CNNs train faster and require significantly fewer computational resources.
- ViTs, while powerful, are more suitable for high-resolution inputs and benefit greatly from large-scale pretraining.
- For low-resolution datasets like CIFAR-10, CNNs remain the more computationally practical option.

Attention vs Spatial Modeling ViTs offer flexibility with global attention, modeling inter-patch relationships directly. This is powerful for capturing global context but can lead to overfitting on small datasets. CNNs leverage spatial hierarchies and are inductively biased for vision tasks, making them highly efficient and effective on structured grid data.

5 Conclusion

In conclusion, CNNs remain a robust choice for image classification, especially for datasets like CIFAR-10 with limited resolution and sample size. ViTs show promise, especially with pretraining and fine-tuning, but require more compute and data. The results suggest that while ViTs are a forward-looking technology, CNNs still dominate in resource-constrained and small-data environments.

References

1. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative Adversarial Networks. In: Advances in Neural Information Processing Systems 27 (2014)
2. Kingma, D.P., Welling, M.: Auto-Encoding Variational Bayes. In: International Conference on Learning Representations (2014)
3. Zhu, J.Y., Park, T., Isola, P., Efros, A.A.: Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks. In: IEEE International Conference on Computer Vision (ICCV) (2017)
4. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention Is All You Need. In: Advances in Neural Information Processing Systems 30 (2017)
5. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N.: An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In: International Conference on Learning Representations (2021)
6. Krizhevsky, A.: Learning Multiple Layers of Features from Tiny Images. Technical Report (2009)
7. Person Face Sketches Dataset. <https://www.kaggle.com/datasets/almightyj/person-face-sketches>
8. Parallel Corpus for English-Urdu Language. <https://www.kaggle.com/datasets/zainuddin123/parallel-corpus-for-english-urdu-language>
9. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324 (1998)
10. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B.: Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. In: IEEE/CVF International Conference on Computer Vision (ICCV) (2021)
11. He, K., Zhang, X., Ren, S., Sun, J.: Deep Residual Learning for Image Recognition. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016)
12. Khan, S., Naseer, M., Hayat, M., Zamir, S. W., Khan, F. S., Shah, M.: Transformers in Vision: A Survey. *ACM Computing Surveys (CSUR)*, 54(10), 1–41 (2021)
13. Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., Jégou, H.: Training data-efficient image transformers distillation through attention. In: *International Conference on Machine Learning (ICML)* (2021)
14. Tan, M., Le, Q.: EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In: *Proceedings of the 36th International Conference on Machine Learning (ICML)* (2019)